The Beginner Guide to Perfect Git Pull Requests

Pull requests are how developers share code updates and ask teammates for feedback. Mastering simple Git workflow habits makes your code reviews faster and helps you ship features without stress.

What you'll learn:

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Cleaning Up Messy Commits with Interactive Rebase

When building a new feature, you often make quick commits like 'temp fix' or 'fixed bug again'. Before opening a pull request, combine those messy commits into one clean message. Interactive rebase lets you pick, squash, or reword your recent commits so your team gets a clean, professional history.

```
$git rebase -i HEAD~3
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Syncing Your Branch to Prevent Merge Conflicts

Main branches change quickly as your teammates merge their work. If you build on an outdated branch, you will get merge conflicts when creating your PR. Rebasing your branch on top of the latest main branch ensures your code runs smoothly alongside everyone else's latest updates.

```
$git fetch origin
 git rebase origin/main
```

3

TIP 3 OF 4

3. Creating Pull Requests Directly from Your Terminal

Switching between your code editor, terminal, and browser takes time. Using the official GitHub CLI tool allows you to create, view, and merge pull requests straight from your terminal command line. It saves time and automatically links your current branch.

```
$gh pr create --title "feat: add user login page" --body "Implements login UI and validations."
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Reviewing Your Own Changes Before Submitting

Never submit a pull request without reviewing your own changes first. Running a git diff against your target branch helps you catch commented-out code, temporary console prints, or forgotten config files before your team reviews your code.

```
$git diff main...feature-branch
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Keep pull requests small and focused on one single feature or fix.
+ Write clear titles using standard prefixes like feat:, fix:, or docs:.
+ Test your changes locally before tagging team members for review.
+ Add screenshots or short screen recordings for visual UI changes.
+ Respond politely to review comments and explain your design choices.

Watch Out For

! Submitting giant 1,000+ line pull requests that take days to review.
! Ignoring merge conflicts until the day of a production release.
! Leaving debug print statements and commented code in your PR.
! Merging code without running automated tests or local build checks.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh